

CS-401

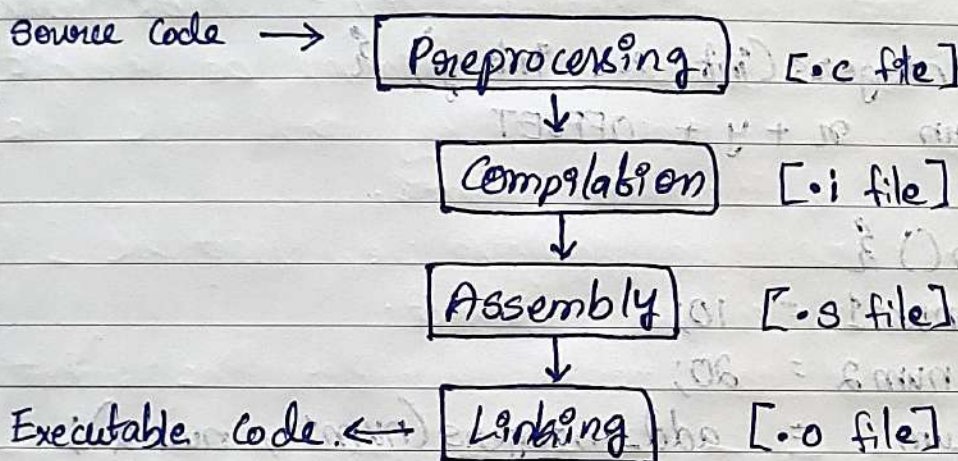
Programming in C

Assignment -1

Q.1.] Explain in details about C-compiler and compilation process with suitable diagrams.

A:• A C-compiler is a program that translates source code written in the C-programming language into binary machine code that a computer can read.

- There are four steps involved in a C-compilation process:



- i.) Preprocessing: The preprocessor takes in the source .c file and performs operations like removing comments, expanding macros, expanding the included files, and conditional compilation to give a '.i file' as output.
- ii.) Compilation: The compiler takes the expanded .i file and converts it into assembly code, which is low-level language and gives a .s file as output.
- iii.) Assembly: The assembler takes the .s file and converts assembly commands into binary code, i.e., .o file.
- iv.) Linking: The linker creates the final executable file in binary by linking object codes of all source files together.

Q.2] Discuss in details about structure of C program. Also write a simple C-program and explain each line of the program.

A: Consider a simple C-program that adds two integers 'x' and 'y' together and prints the sum:

```
/* Simple C-program to add 2 integers */
```

```
#include <stdio.h>
```

```
#define OFFSET 0
```

```
int total_sum = 0;
```

```
int add_integers(int x, int y) {
```

```
    return x + y + OFFSET
```

```
}
```

```
int main() {
```

```
    int num1 = 10;
```

```
    int num2 = 20;
```

```
    total_sum = add_integers(num1, num2);
```

```
    printf("The sum is = %d\n", total_sum);
```

```
    return 0;
```

```
}
```

The Structure of this C-program:

i) Documentation Section: Provides description to the program in the form of comments.

• Here, `/* Simple C-program ... */` is the comment that gives a brief description for the program.

ii) Preprocessor Section: All the header files are declared in this section, which helps us access other's code.

• Here, `#include <stdio.h>` includes standard I/O library into the program.

iii) Definition Section: The `#define` preprocessor is used to create a constant throughout the program.

- Here, `#define OFFSET=0` assigns a symbolic name to a constant whose value is defined to be zero.

iv) Global Declaration Section: This section contains global variables, function declaration and static vars.

- Here, `int total-sum = 0` sets the value of the global variable 'total-sum' to zero.

v) Main () function: Every C program must have a `main()` function. Operations, declarations and executions are performed inside it. It can return `int` as value.

- Here, `int main() {` → declares the `main()` function.

`int num1 = 10;` → initializes `num1` to 10.

`int num2 = 20;` → initializes `num2` to 20.

`total-sum = add-integers(num1, num2);` → calls the 'add-integers' function by passing `num1` and `num2` as argument and stores the res result in the global variable.

`printf("...")` → prints the result to the console.

`return 0` → indicates that the function passed successfully.

vi) Sub Programs: This section contains user defined functions. These functions are called as required.

- Here, `int add-integers(int x, int y) {` → defines the function taking two `int` values as arguments.

`return x+y+OFFSET` → performs addition on `x` & `y` and `OFFSET` and returns the result.

Q.3] What are C identifiers? Discuss the Rules for naming identifiers with suitable examples.

A: In C programming, identifiers are names given to various program elements like variables, functions, arrays, structures and labels to uniquely identify them during program execution.

• Rules for naming identifiers in C:

- i) Identifiers can only contain uppercase and lowercase letters (a-z & A-Z), digits (0-9) and underscore (_).
- ii) First character in an identifier must start with a letter or an underscore. It cannot be a digit.
- iii) You cannot use reserved keywords in C like 'int', 'return', etc as identifier names.
- iv) Identifiers are case sensitive; i.e., uppercase & lowercase letters are treated as distinct.
- v) No whitespaces or special characters like !, @, #, allowed

• Valid identifier names	• Invalid identifier names
→ total_sum ✓, num_1 ✓	→ 1num_X, total sum_X
→ MAX_SIZE	→ MAX-SIZE_X
→ integer1 ✓, taxes ✓	→ int_X, tax \$X

Page No.:
Date:

Q.4] What is Data Types in C? Explain with various data types in details with suitable examples.

A:- In C programming, a data type specifies the type and size of data that a variable can hold and how the compiler handles the data. They define how the memory will be allocated to the data and what operations can be performed on them.

- Data types are broadly divided into 3 distinct categories:

I.] Basic Data Types - built-in in C, foundational.

- int : stores integers, size is 4 bytes, eg: `int age = 23;`
- char : stores single character, size 1 byte, eg: `char grade = 'A';`
- float : stores single-precision decimals, 4 bytes, eg: `float price = 10.50;`
- double : stores double-precision decimals, 8 bytes, eg: `double pi = 3.14159;`

II.] Derived Data Types - created by building upon basic data types.

- Arrays : sequentially stored elements of same data type.
(eg: `int scores[3] = {95, 88, 72};`)
- Pointers : stores memory address of another variable.
(eg: `int num = 5; int *ptr = #`)
- Functions : Accepts arguments and returns a value.
(eg: `int add(int a, int b) { return a+b; }`)

III.] User-Defined Data Types - Allows to create custom structures.

- structs : groups variables of different data types under ^{single} name.
(eg: `struct Student { char name[20]; int rollNo; }`)
- union : internal members share same memory location.
(eg: `union Packet { int dataInt; char dataChar; }`)
- enum : assigns human readable named integer constants.
(eg: `enum Day { MON, TUE, WED, THUR, FRI, SAT, SUN };`)

Q.5.] Discuss operators in C with suitable examples.

A:.. An operator in C is a symbol that tells the compiler to perform mathematical, logical or data manipulations on values called operands.

- Operands in C can be variables, constants or expressions.
- Operators can be ~~from~~ unary, binary or ternary based on the number of operands.

1. Arithmetic Operators: (if $a = 10$ and $b = 4$)

i) Addition $\rightarrow a + b = 14$

ii) Subtraction $\rightarrow a - b = 6$

iii) Multiplication $\rightarrow a * b = 40$

iv) Division $\rightarrow a / b = 2$
(Returns integer quotient)

v) Modulus $\rightarrow a \% b = 2$
(Returns division remainder)

2. Relational Operators: (if $a = 10$ and $b = 4$)

i) Equal to $\rightarrow a == b \rightarrow 0$ (False)

ii) Not equal to $\rightarrow a != b \rightarrow 1$ (True)

iii) Greater than $\rightarrow a > b \rightarrow 1$ (True)

iv) Less than $\rightarrow a < b \rightarrow 0$ (True)

v) Greater than or equal to $\rightarrow a >= b \rightarrow 1$ (True)

vi) Less than or equal to $\rightarrow a <= b \rightarrow 0$ (False)

3. Logical Operators:

i) Logical AND $\rightarrow \&\&$ (Returns True if both operands are True)

ii) Logical OR $\rightarrow \|\|$ (Returns True if at least one operand is True)

iii) Logical NOT $\rightarrow !$ (Reverses the logical state)

4. Assignment Operators:

- i.) Simple assignment $\rightarrow a = b$ (assigns value of b to a)
- ii.) Add and assign $\rightarrow a += b$ (same as $a = a + b$)
- iii.) Multiply and assign $\rightarrow a *= b$ (same as $a = a * b$)
- iv.) Minus and assign $\rightarrow a -= b$ (same as $a = a - b$)
- v.) Divide and assign $\rightarrow a /= b$ (same as $a = a / b$)

5. Bitwise Operators:

- i.) Bitwise AND $\rightarrow \&$ (copies a bit if exists in both operands)
- ii.) Bitwise OR $\rightarrow |$ (copies a bit if exists in either operands)
- iii.) Bitwise XOR $\rightarrow ^$ (copies a bit if set in only one operand)
- iv.) Left Shift $\rightarrow <<$ (shifts bits left, multiplying by 2 per shift)
- v.) Right Shift $\rightarrow >>$ (shifts bits right, multiplying by 2 per shift)

6. Other Operators:

- i.) Increment $\rightarrow a++$ (same as $a = a + 1$)
- ii.) Decrement $\rightarrow a--$ (same as $a = a - 1$)
- iii.) Size of $\rightarrow \text{sizeof}(\text{int}) = 4$ (returns data size in bytes)
- iv.) Comma operator $\rightarrow ,$
- v.) Conditional operator $\rightarrow \text{condition} ? \text{expr1} : \text{expr2}$ (shorthand for if else)
- vi.) Arrow operator $\rightarrow \rightarrow$ (used to reference members of structs/unions)
- vii.) Address of $\rightarrow \&$ (gets the address of a variable)
- viii.) Pointer operator $\rightarrow *$ (points to an address variable)